UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO DO UÍGE
CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA
SISTEMA DE GESTÃO DE BASE DE DADOS II

2026

OTIMIZAÇÃO DE ROTAS COM GRAFOS (ALGORITMO DE DIJKSTRA)

RELATÓRIO TÉCNICO

POR:

FRANCISCO BÁRULIO FIGUEIREDO ANTÓNIO, JOÃO JOSÉ ANTÓNIO DA SILVA e KUMBI MANUEL DOMINGOS

Startup: TAKETWARE

Tecnológias: Pycharm

ORIENTADOR

Moyo Kanivengidio, MSc.

Sumário

ENQUADRAMENTO TEÓRICO 4

1. **Algoritmo de Dijkstra** 4
2. **Grafos** 4

IMPLEMENTAÇÃO DO ALGORITMO 5

- **Implementação do grafo conexo ponderado:** 5
- **Implementação do Algoritmo de Dijkstra** 6
- **Implementação do programa main.py para realizar os teste** 8

CONCLUSÃO 10

INTRODUÇÃO

O trabalho em questão tem como objetivo principal apresentar um algoritmo de Otimização de rotas com grafos (Algoritmo de Dijkstra) desenvolvendo em Python um programa que calcula a melhor rota para sair de um determinado ponto (origem) para se chegar em um ponto final (destino). Será apresentado o funcionamento do programa para o entendimento e aprendizado do algoritmo de Dijkstra e grafos para implementação deste projeto.

ENQUADRAMENTO TEÓRICO

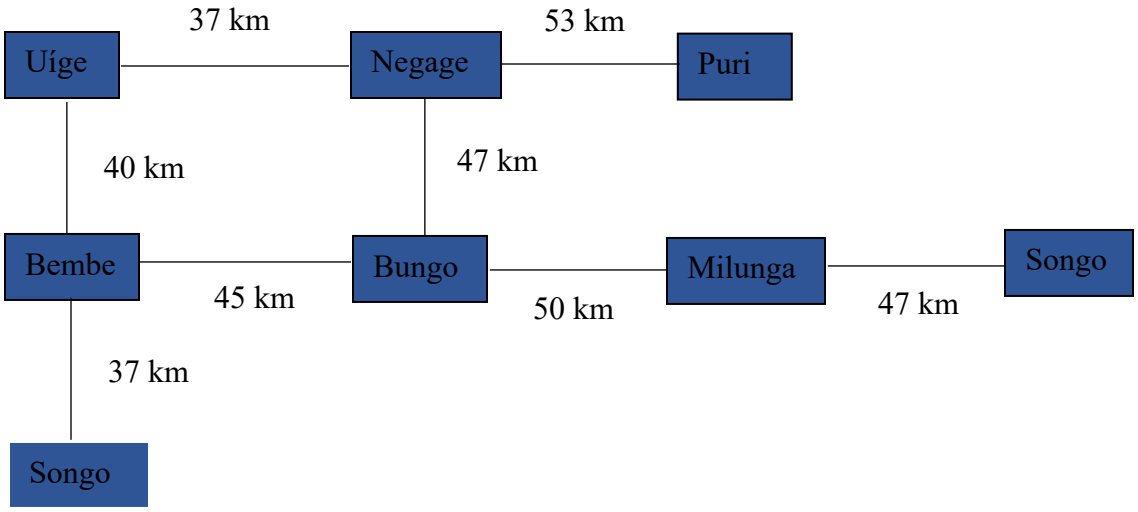
1. Algoritmo de Dijkstra

O algoritmo de Dijkstra é um algoritmo de caminho mínimo usado em grafos. Ele é usado para encontrar o caminho mais curto entre dois vértices em um grafo ponderado, onde os pesos são associados às arestas.

2. Grafos

Um grafo é uma estrutura, mais precisamente uma abstração matemática que modela relações dois a dois entre determinados elementos de um conjunto. Existem váriados tipos de grafos, dos quais são: grafos simples, direcionados, não direcionados, ponderados, completos, conexos e entre outros, dos quais, eles podem ser classificados por Arestas, Peso e Rotulagem, Estrutura e conexão.

Para o nosso trabalho nós fizemos o uso de coneceitos de grafos ponderados (classificação por Peso e rotulagem) e grafo conexo (classificação por estrutura e conexão), abaixo encontra-se a representação do grafo usado como mapa neste nosso projeto:



IMPLEMENTAÇÃO DO ALGORITMO

Cada caminho num grafo ponderado possui um peso (a soma dos pesos das arestas que o constituem). Esta característica nos possibilita fazer a implementação do algoritmo de Dijkstra para encotrar o menor caminho possível.

- **Implementação do grafo conexo ponderado:**

```
class GrafoPonderado:

    def __init__(self):
        self.adjacencia = {}

    def adicionar_no(self, no): #aqui nós começamos por criar o grafo
        if no not in self.adjacencia: #nó adjacente
            self.adjacencia[no] = []

    def adicionar_aresta(self, origem, destino, peso): # Aqui começamos
por adicionar as arestas
        #E em relação aos nós, caso não existirem serão criados
automaticamente
        if peso < 0: #Como o algoritmo de Dijkstra não suporta valores
negativos, este passo serve para verificar e informar
            raise ValueError(
                f"Peso negativo ({peso}) não é suportado pelo algoritmo
de Dijkstra."
            )

        self.adicionar_no(origem) #Adiciona o nó de origem
        self.adicionar_no(destino) #Adiciona o nó de destino
        self.adjacencia[origem].append((destino, peso)) #Estabelece o
peso entre as arestas
        self.adjacencia[destino].append((origem, peso)) # Aqui também é
estabelecido o peso na aresta

# Aqui fazemos a validação da criação do nosso grafo

    def no_existe(self, no): #Aqui procuramos verificar se o nó existe no
grafo criado
        return no in self.adjacencia

    def nos(self): #Aqui fazemos o retorno, listando os elementos do
nosso grafo
        return list(self.adjacencia.keys())

    def total_nos(self): # Aqui procuramos mostrar o número de nós que
existem no nosso grafo
        return len(self.adjacencia)

    def total_arestas(self): # Procuramos ser claros, mostrando também o
número de arestas que tem o nosso grafo
        return sum(len(v) for v in self.adjacencia.values()) // 2

# Aqui fazemos a exibição do grafo

    def exibir(self): # Aqui imprimimos a lista dos componentes do grafo
```

```
de maneira formatada
        print("\nLista de Adjacência do Grafo")
        print(" " * 48)
        for no, vizinhos in self.adjacencia.items():
            if vizinhos:
                ligacoes = ",  ".join(f"{v} ({p} km)" for v, p in
vizinhos)
            else:
                ligacoes = "(nó isolado)"
            print(f"  {no:12} →  {ligacoes}")
        print(f"\n  Nós: {self.total_nos()}   |   Arestas:
{self.total_arestas()}")
        print(" " * 48)
```

- **Implementação do Algoritmo de Dijkstra**

```
import heapq # Usando a lógica de complexidade de algoritmos, decidimos
importar o módulo nativo que permite
             # trabalhar com prioridade em filas permitibdo encontrar os
menores ou maiores elementos de uma coleção.

def dijkstra(grafo, origem): #Aqui começamos por implementar o algoritmos
de Dijkstra

    if not grafo.no_existe(origem): #Aqui decidimos fazer a verificação
da existência do nó no grafo
                                    #Algo similar com o BREAK, aqui caso
for executado, o fluxo do progama é interrompido
        raise ValueError(f"Nó de origem '{origem}' não existe no grafo.")

    distancias = {no: float('inf') for no in grafo.adjacencia} #Aqui,
pensamos em inicializar com valores bastante grandes
                                                                # ou seja,
com infinito
    distancias[origem] = 0 # A distância da origem continua sendo zero
    predecessores = {no: None for no in grafo.adjacencia} # Aqui
procuramos registar cada nó no caminho ótimo
    fila = [(0, origem)] # Aqui fica armazenado a distância entre os nós
quando o algoritmo percorre os nós

    # Conjunto de nós já finalizados (distância mínima confirmada)
    visitados = set() #Aqui ficam armazenados os nós percorridos (apenas
os do caminho ótimo)

    while fila:

        dist_atual, no_atual = heapq.heappop(fila)
        if no_atual in visitados: # Aqui fazemos os testes entre as
distâncias, onde se já foi processado com a menor distância, ignora
            continue
        visitados.add(no_atual)
        for vizinho, peso in grafo.adjacencia[no_atual]: #Ignora as
arestas dos nós vizinhos
            nova_dist = dist_atual + peso

            if nova_dist < distancias[vizinho]:
```

```
                distancias[vizinho] = nova_dist
                predecessores[vizinho] = no_atual
                heapq.heappush(fila, (nova_dist, vizinho))

    return distancias, predecessores

#Aqui é onde fazemos a renconstrunção dos caminhos percorridos
def reconstruir_caminho(predecessores, origem, destino):
    if destino not in predecessores: #Aqui verificamos de o destino
existe no grafo criado
        return []

    caminho = []
    no_atual = destino

    while no_atual is not None:
        caminho.append(no_atual)
        no_atual = predecessores[no_atual]

    caminho.reverse()

    if caminho[0] != origem: #Aqui verificamos se o caminho não começa na
origem, logo o destino é inalcançável
        return []

    return caminho

#Aqui criamos funções que nos irão permitir exibir
def exibir_caminho(grafo, origem, destino):
    """
    Calcula e exibe o caminho mais curto entre dois nós,
    tratando todos os casos especiais (nó inexistente, inalcançável).

    Parâmetros:
        grafo   — instância de GrafoPonderado
        origem  — nó de partida
        destino — nó de chegada
    """
    print(f"\nOrigem: {origem}  →  Destino: {destino}")
    print("—" * 42)

    if not grafo.no_existe(origem): # Antes de calcular os nós,
verificamos se eles existem
        print(f"Erro: o nó '{origem}' não existe no grafo.")
        print("—" * 42)
        return

    if not grafo.no_existe(destino):
        print(f"Erro: o nó '{destino}' não existe no grafo.")
        print("—" * 42)
        return

    if origem == destino:
        print(f"Origem e destino são o mesmo nó. Distância: 0 km.")
        print("—" * 48)
        return
```

```
    distancias, predecessores = dijkstra(grafo, origem)
    caminho = reconstruir_caminho(predecessores, origem, destino)

    if not caminho:
        print(f"Não existe caminho entre '{origem}' e '{destino}'.")
        print(f"(Nós não conectados no grafo.)")
    else:
        rota = " → ".join(caminho)
        total = distancias[destino]
        print(f"  Caminho   : {rota}")
        print(f"  Distância : {total} km")

    print("—" * 42)


def exibir_todas_distancias(grafo, origem): #Aqui exibimos as distâncias

    if not grafo.no_existe(origem): # Auto explicativo, mas em síntese
decidmos verificar a existência do nó no grafo
        print(f"\nNó '{origem}' não existe no grafo.")
        return

    distancias, _ = dijkstra(grafo, origem)

    print(f"\nDistâncias mínimas a partir de '{origem}'")
    print("—" * 42)

    for cidade, dist in sorted(distancias.items(), key=lambda x: x[1]):
#Aqui decidimos organizar os elementos de forma ascendente
        if dist == float('inf'):
            print(f"  {cidade:14} :  inalcançável")
        else:
            print(f"  {cidade:14} : {dist:3} km  {barra}")

    print("—" * 42)
```

- **Implementação do programa main.py para realizar os teste**

```
from grafo import GrafoPonderado
from dijkstra import exibir_caminho, exibir_todas_distancias
#Definição do nosso grafo
def criar_grafo():
    """
    Criamos os nosso grafo com 7 nós (municípios do Uíge)

    Estrutura visual do grafo:

        Uíge ——37—— Negage ——53—— Puri
          |             |
         40            47
          |             |
        Bembe ——45—— Bungo ——50—— Milunga ——47—— Songo
          |
         37
          |
        Songo
```

```
    Arestas e pesos (distâncias em km):
        Uíge   — Negage  :  37 km
        Negage — Puri    :  53 km
        Uíge   — Bembe   :  40 km
        Negage — Bungo   :  47 km
        Bembe  — Bungo   :  45 km
        Bungo  — Milunga :  50 km
        Milunga— Songo   :  47 km
        Bembe  — Songo   :  37 km
    """
    g = GrafoPonderado()

    g.adicionar_aresta("Uíge",    "Negage",   37)
    g.adicionar_aresta("Negage",  "Puri",     53)
    g.adicionar_aresta("Uíge",    "Bembe",    40)
    g.adicionar_aresta("Negage",  "Bungo",    47)
    g.adicionar_aresta("Bembe",   "Bungo",    45)
    g.adicionar_aresta("Bungo",   "Milunga",  50)
    g.adicionar_aresta("Milunga", "Songo",    47)
    g.adicionar_aresta("Bembe",   "Songo",    37)

    return g


#Algo similar na linguagem C, aqui demos o início do nosso programa
principal
def main():
    print()
    print("  " + "=" * 48)
    print("   OTIMIZAÇÃO DE ROTAS — ALGORITMO DE DIJKSTRA")
    print("=" * 42)

    grafo = criar_grafo() # Aqui criamos o grafo principal
    grafo.exibir() # Aqui exibimos a estrutura do grafo

# Zona Para testes

    exibir_caminho(grafo, "Bungo", "Milunga")

if __name__ == "__main__":
    main()
```

CONCLUSÃO

O desenvolvimento deste trabalho foi rico em aprendizagem. A linguagem Python é muito vasta, que permitiu progredir com o trabalho, este trabalho requereu uma constante pesquisa de materiais como livros, e um acesso constante a internet para satisfazer as dificuldades que iam aparecendo. Foram estudados conteúdos desde conceitos básicos e intermédios da liguagem Python e estrutura de dados (árvores e grafos) bem como algoritmos (algoritmo de Dijkstra) para se puder concluir e fazer a entrega do trabalho em conformidade.